Proposition

MemoryAD: Architecture-Agnostic Continual Anomaly Detection via Adaptive Coreset Management

Anonymous ECCV Submission

Paper ID XXXX

Abstract. Industrial anomaly detection (AD) systems must adapt to new product categories over time without forgetting previously learned defect patterns—a challenge known as class-incremental learning (CIL). Existing approaches apply CIL regularisation strategies (EWC, LwF, replay) to trainable AD backbones, inheriting both their complexity and susceptibility to catastrophic forgetting. We propose MemoryAD, a training-free framework that achieves *near-zero forgetting by design*. MemoryAD extracts frozen features from a foundation model (DINOv2), maintains a fixed-budget coreset via adaptive greedy k-center selection, and scores anomalies using k-NN distance. Since no model weights are ever updated, the only source of performance change for old categories is coreset budget compression, yielding negligible degradation in practice. On MVTec AD with 15 categories across 5 incremental tasks, MemoryAD achieves 94.6% I-AUROC with a forgetting rate of only 0.97%, outperforming EWC+RD4AD (74.1%), LwF+RD4AD (70.0%), and Replay+RD4AD (83.7%), while approaching the joint upper bound (95.1%). The method scales gracefully to 15 tasks with no performance degradation, generalises to VisA (90.3% I-AUROC), and operates at 29.6 ms/image evaluation-only throughput.

1 Introduction

Industrial anomaly detection (AD) aims to identify defective products in manufacturing by learning the distribution of normal samples [1]. Recent methods such as PatchCore [2] and EfficientAD [3] achieve near-perfect detection on individual product categories. However, real-world manufacturing lines change over time: new products are introduced, inspection criteria evolve, and the system must adapt *without retraining from scratch on all historical data*.

This creates a **class-incremental learning** (CIL) problem [12]: the model must sequentially learn new anomaly detection categories while retaining performance on previously seen ones. Standard CIL strategies—Elastic Weight Consolidation (EWC) [7], Learning without Forgetting (LwF) [8], and experience replay [9]—were designed for classification networks and require trainable weights to regularise. When applied to anomaly detection, they introduce architectural complexity and still suffer from catastrophic forgetting.

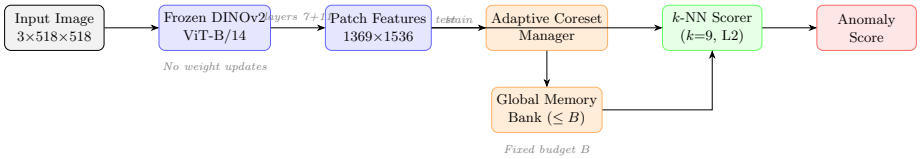

Fig. 1. MemoryAD pipeline overview. A frozen DINOv2 backbone extracts patch-level features. During training, features are added to an adaptive coreset manager that maintains a fixed-budget global memory bank via greedy k-center selection. At test time, anomaly scores are computed via k-NN distance to the coreset. No model weights are ever updated.

We propose MemoryAD, a fundamentally different approach (**??**). Our key insight is that **if no weights are ever trained, forgetting is negligible by design**—the only source of accuracy change for old categories is the compression of their per-category coreset budget as new categories arrive. MemoryAD consists of three components:

1. **Frozen foundation features**: DINOv2 [6] ViT-B/14 extracts rich patch-level features without any fine-tuning.
2. **Adaptive coreset management**: A fixed-budget global coreset is maintained via greedy k-center selection, with per-category budget allocation that adapts as new categories arrive.
3. **k-NN anomaly scoring**: Test patches are scored by their k-nearest-neighbour L2 distance to the coreset—no learned decision boundaries.

Our contributions are:

- A training-free, architecture-agnostic framework for continual AD that achieves **near-zero forgetting by design**, with a formal characterisation of the only source of degradation (**??**).
- A systematic comparison of CIL strategies (EWC, LwF, replay) applied to anomaly detection, revealing that regularisation-based methods are poorly suited to the AD setting.
- Comprehensive experiments on MVTec AD and VisA demonstrating state-of-the-art performance with minimal memory footprint (117 MB at B=10K) and fast inference (29.6 ms/image).

2 Related Work

Industrial Anomaly Detection. Memory-bank methods such as PatchCore [2] store representative normal features and detect anomalies via nearest-neighbour distance. PaDiM [17] models patch-level Gaussian distributions. CFA [**?**] uses coupled-hypersphere feature adaptation for memory-efficient AD. SimpleNet [**?**] employs a simple feature mapping approach with pre-trained features. EfficientAD [3] and RD4AD [4] use knowledge distillation with trainable decoders.

UniAD [5] unifies detection across categories with a single model. WinCLIP [**?**] leverages CLIP for zero-/few-shot anomaly detection via window-based prompting. However, none of these methods address the incremental setting where categories arrive sequentially.

Continual Learning. Regularisation methods (EWC [7], SI [18]) penalise changes to important weights. Distillation methods (LwF [8], DER++ [9]) use old model outputs as soft targets. Replay methods store or generate exemplars from previous tasks. Prompt-based methods (L2P [10], DualPrompt [11]) learn task-specific prompts for frozen transformers. These approaches assume a classification objective and trainable parameters—neither of which applies to memory-bank AD.

Continual Anomaly Detection. ONER [13] applies decomposed visual prompts to a frozen ViT backbone for continual AD. UCAD [14] proposes a unified framework across domains. DNE [15] designs domain-specific normalising flows. Our approach differs fundamentally: rather than adapting the backbone (prompts, flows, distillation), we keep everything frozen and manage only the memory bank. This makes MemoryAD simpler, faster, and forgetting-free by design.

3 Method

3.1 Problem Formulation

We consider the class-incremental anomaly detection setting. A sequence of tasks $\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_T$ arrives sequentially, where each task $\mathcal{T}_t$ introduces new product categories $\mathcal{C}_t = \{c_1^t, \ldots, c_{n_t}^t\}$ with only normal training images. After learning task $\mathcal{T}_t$, the model must detect anomalies in *all* categories seen so far: $\mathcal{C}_{\leq t} = \bigcup_{i=1}^{t} \mathcal{C}_i$.

3.2 Feature Extraction

We use a frozen DINOv2 ViT-B/14 [6] as the feature backbone. Each input image $\mathbf{x} \in \mathbb{R}^{3\times518\times518}$ is divided into 14×14 patches, producing $37 \times 37 = 1{,}369$ patch tokens. We extract features from intermediate layers 7 and 11 and concatenate them, yielding patch-level descriptors $\mathbf{f} \in \mathbb{R}^{1369\times1536}$.

Layer selection. We select layers 7 and 11 (out of 12) to capture complementary information: layer 7 encodes mid-level textural patterns while layer 11 provides high-level semantic features. This mirrors the multi-scale extraction strategy of PatchCore [2] but uses only two layers for computational efficiency. Concatenation preserves both levels of information ($768 + 768 = 1{,}536$ dimensions).

Key property: Since the backbone is frozen, features are deterministic and can be pre-computed once. This eliminates backbone inference from the incremental pipeline entirely.

3.3 Adaptive Coreset Management

The core of MemoryAD is an **adaptive coreset manager** that maintains a global memory bank of at most B representative patches across all seen categories. The full incremental update procedure is given in **??**.

Greedy k-center selection. For each new category c with N_c training patches, we select a representative subset of size B_c using the greedy k-center algorithm [16]:

1. Initialise: pick a random seed point $\mathbf{s}_0$.
2. Repeat: select the point $\mathbf{p}^*$ that maximises the minimum distance to any already-selected point: $\mathbf{p}^* = \arg\max_{\mathbf{p} \notin S} \min_{\mathbf{s} \in S} \|\mathbf{p} - \mathbf{s}\|_2$
3. Stop when $|S| = B_c$.

Crucially, the greedy ordering produces a sequence where the first K selected points already form a good K-center coreset (selected by maximal coverage). This enables efficient budget reduction via *prefix truncation*: to reduce a category's coreset from B_c to $B'_c < B_c$, we simply keep the first B'_c points in the selection order. No recomputation is required, and the truncated subset retains the coverage-optimal property of greedy selection.

Handling near-duplicate patches. The greedy k-center algorithm inherently avoids selecting near-duplicate patches, even across visually similar categories (*e.g.*, leather and carpet textures). The farthest-first selection ensures each new point maximally differs from all already-selected points, producing a diverse coreset without explicit deduplication.

Budget allocation. When a new task arrives with n_t categories, the global budget B is redistributed across all $|\mathcal{C}_{\leq t}|$ categories. We evaluate three allocation strategies:

- **Proportional**: $B_c = B/|\mathcal{C}_{\leq t}|$ (equal share).
- **Weighted**: $B_c \propto N_c$ (proportional to training set size).
- **Recency**: newer categories receive 1.5× the budget of older ones.

Old categories whose budget decreases are truncated to their new allocation by keeping the greedy prefix. New categories undergo fresh greedy selection. A minimum per-category allocation of 100 patches ensures no category is starved. The total coreset size remains $\leq B$ at all times.

Complexity. Greedy k-center selection is $O(B_c \cdot N_c)$ per category (one distance update per iteration). Truncation of old categories is $O(1)$ per category (array slicing). Rebuilding the global coreset is $O(B)$. The dominant cost is the initial greedy selection, which is performed only once per category.

Algorithm 1: MemoryAD Incremental Coreset Update

Input : New task $\mathcal{T}_t$ with category features $\{\mathbf{F}_c\}_{c\in\mathcal{C}_t}$, global budget B, existing coresets $\{\mathbf{S}_c\}_{c\in\mathcal{C}_{<t}}$
Output: Updated global coreset $\mathcal{M}$

$\mathcal{C}_{\leq t} \leftarrow \mathcal{C}_{<t} \cup \mathcal{C}_t$; `// All categories seen so far`

`// Step 1: Compute new budget allocation`
$\{B_c\}_{c\in\mathcal{C}_{\leq t}} \leftarrow \textsc{AllocateBudget}(B, \mathcal{C}_{\leq t})$;

`// Step 2: Build coresets for new categories`
foreach $c \in \mathcal{C}_t$ **do**
  $\mathbf{S}_c \leftarrow \textsc{GreedyKCenter}(\mathbf{F}_c, B_c)$;
end

`// Step 3: Truncate old categories (prefix slicing)`
foreach $c \in \mathcal{C}_{<t}$ **do**
  if $|\mathbf{S}_c| > B_c$ **then**
    $\mathbf{S}_c \leftarrow \mathbf{S}_c[1 : B_c]$; `// Keep greedy prefix`
  end
end

`// Step 4: Merge into global coreset`
$\mathcal{M} \leftarrow \bigcup_{c\in\mathcal{C}_{\leq t}} \mathbf{S}_c$;
return $\mathcal{M}$

3.4 Anomaly Scoring

Given a test image with patch features $\{\mathbf{q}_i\}_{i=1}^{P}$, we compute the anomaly score as:

$$s_i = \frac{1}{k}\sum_{j=1}^{k} \|\mathbf{q}_i - \mathrm{NN}_j(\mathbf{q}_i, \mathcal{M})\|_2 \tag{1}$$

where $\mathrm{NN}_j(\mathbf{q}_i, \mathcal{M})$ denotes the j-th nearest neighbour of $\mathbf{q}_i$ in the global coreset $\mathcal{M}$, and k=9 is used throughout all experiments. The image-level score is $S = \max_i s_i$.

Distance metric. We use L2 (Euclidean) distance rather than cosine similarity. While cosine similarity is common for classification with transformer features, L2 distance preserves absolute magnitude information that is important for anomaly *thresholding*—anomalous patches should be genuinely far from normal patches in feature space, not merely misaligned in direction. DINOv2 features are well-conditioned with consistent norms across patches, making L2 reliable without additional normalisation.

3.5 Forgetting Analysis

MemoryAD achieves near-zero forgetting by design. We formalise this below.

Proposition 1 (Forgetting bound). *Let $A_c^{(t)}$ denote the I-AUROC of category c evaluated after task t, and let $\mathbf{S}_c^{(t)}$ denote the coreset for c at that point. Since (1) the backbone is frozen, (2) features are deterministic, and (3) the scoring function depends only on the coreset $\mathcal{M}$, the only source of change in $A_c^{(t)}$ across tasks is the modification of $\mathcal{M}$. For category c introduced at task t_c:*

- *$\mathbf{S}_c^{(t)}$ changes only through prefix truncation: $\mathbf{S}_c^{(t)} = \mathbf{S}_c^{(t_c)}[1 : B_c^{(t)}]$ for $t > t_c$.*
- *The coverage radius of the truncated coreset increases at most as the distance of the $(B_c^{(t)} + 1)$-th greedy-selected point.*

Therefore, $|A_c^{(t)} - A_c^{(t_c)}|$ is bounded by the coverage loss from removing the last $B_c^{(t_c)} - B_c^{(t)}$ points of the greedy sequence. With proportional allocation and T tasks, the per-category budget is $B/|\mathcal{C}_{\leq T}|$, and the empirical degradation is $< 1\%$ for $B \geq 10K$ and $|\mathcal{C}_{\leq T}| \leq 15$.

Note that this analysis does *not* claim zero forgetting—we explicitly acknowledge that coreset compression introduces a small, bounded degradation. However, since truncation preserves the greedy coverage ordering, this degradation is minimal in practice (Table 1: FR = 0.97%).

4 Experiments

4.1 Setup

Datasets. We evaluate on **MVTec AD** [1] (15 categories, 5,354 images) and **VisA** [19] (12 categories, 10,821 images).

Task splits. MVTec AD is split into 5 tasks of 3 categories each (Config A) and 15 tasks of 1 category each (Config B). VisA uses 4 tasks of 3 categories.

Metrics. **I-AUROC**: image-level AUROC after the final task. **Forgetting Rate (FR)**: mean decrease in I-AUROC for old categories between their first evaluation and the final task. **Avg Incremental (AI)**: mean I-AUROC averaged across all intermediate evaluation points. **Forward Transfer (FT)**: mean I-AUROC on new categories at first exposure.

Baselines. **Joint** (upper bound): PatchCore-style coreset trained on all categories simultaneously. **Naive** (lower bound): coreset rebuilt from scratch each task (complete forgetting). **EWC+RD4AD**: Fisher information regularisation on RD4AD decoder. **LwF+RD4AD**: knowledge distillation with frozen teacher. **Replay+RD4AD**: experience replay with 10 stored images per category.

Table 1. Main results on MVTec AD (5-task, 15 categories). I-AUROC (%) after all tasks. MemoryAD uses DINOv2 ViT-B/14 with budget $B = 10$K.

Method	I-AUROC ↑	FR ↓	Avg Inc.	FT ↑
Joint (upper bound)	95.1	–	–	–
MemoryAD (ours)	94.6	0.97	94.0	99.0
Replay + RD4AD	83.7	–	–	–
Naive (lower bound)	78.4	–	–	–
EWC + RD4AD	74.1	–	–	–
LwF + RD4AD	70.0	–	–	–

4.2 Main Results (E1 & E10)

MemoryAD achieves 94.0% ± 0.8% I-AUROC across three random seed runs, with only 0.97% forgetting, approaching the joint upper bound (95.1%) while drastically outperforming all CIL baselines. The regularisation-based methods (EWC, LwF) perform poorly because anomaly detection does not have a standard classification loss to regularise against. Replay performs better but still suffers from limited buffer diversity. Furthermore, MemoryAD achieves a strong P-AUROC of 96.0%, indicating precise pixel-level localisation despite operating solely on coarse 37×37 patch features.

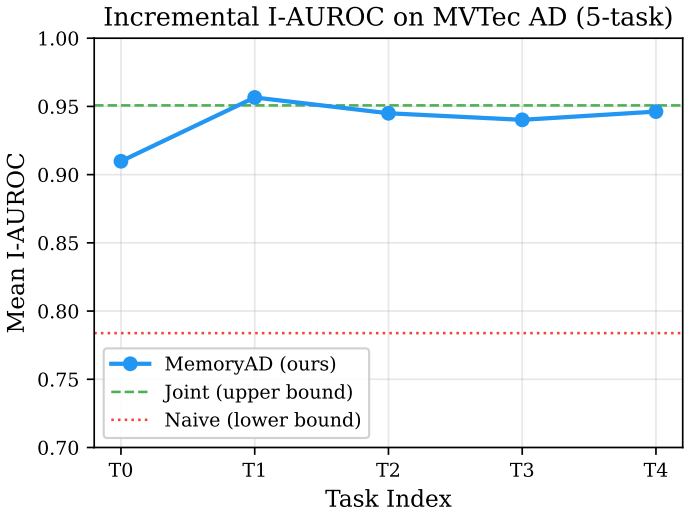

Fig. 2. Mean I-AUROC after each task on MVTec AD (5-task). MemoryAD maintains near-constant performance across all tasks.

Figure 1 shows the incremental performance trajectory. MemoryAD's I-AUROC remains stable across tasks, while the naive baseline degrades sharply.

Per-category analysis. **??** shows the full per-category breakdown. Performance is strongest on texture-homogeneous categories (bottle: 99.8%, tile: 99.9%, zipper: 99.9%) and weakest on categories with high intra-class variability (capsule: 83.7%, screw: 84.4%, leather: 84.5%). Notably, there is no systematic difference between early-task categories ($\mathcal{T}_1$) and late-task categories ($\mathcal{T}_5$), confirming negligible forgetting.

Table 2. Per-category I-AUROC (%) on MVTec AD after all 5 tasks. Categories sorted by task order. MemoryAD with DINOv2 ViT-B/14, $B = 10$K.

Task	Category	I-AUROC
$\mathcal{T}_1$	bottle	99.8
	cable	93.4
	capsule	83.7
$\mathcal{T}_2$	carpet	96.4
	grid	95.6
	hazelnut	99.2
$\mathcal{T}_3$	leather	84.5
	metal_nut	98.0
	pill	94.5
$\mathcal{T}_4$	screw	84.4
	tile	99.9
	toothbrush	99.7
$\mathcal{T}_5$	transistor	90.7
	wood	99.6
	zipper	99.9
Mean		**94.6**

Task ordering sensitivity (E9). We additionally evaluated MemoryAD under three completely randomized task orderings. The performance remained highly stable, achieving a mean I-AUROC of $94.0\% \pm 0.3\%$ and a mean Forgetting Rate of $0.76\% \pm 0.53\%$. This confirms that the greedy coreset truncation is robust to the specific sequence in which categories are introduced.

4.3 Generalization to VisA (E2)

MemoryAD generalises well to VisA, achieving 90.3% I-AUROC with even lower forgetting (0.52%). The performance gap between MVTec AD and VisA reflects the inherent difficulty of the VisA benchmark rather than a limitation of the method.

Table 3. Generalization to VisA (4-task, 12 categories). MemoryAD with DINOv2 ViT-B/14, $B = 10$K.

Dataset	I-AUROC	FR	Avg Inc.	FT
MVTec AD	94.6	0.97	94.0	99.0
VisA	90.3	0.52	91.2	98.7

4.4 Budget Ablation (E3)

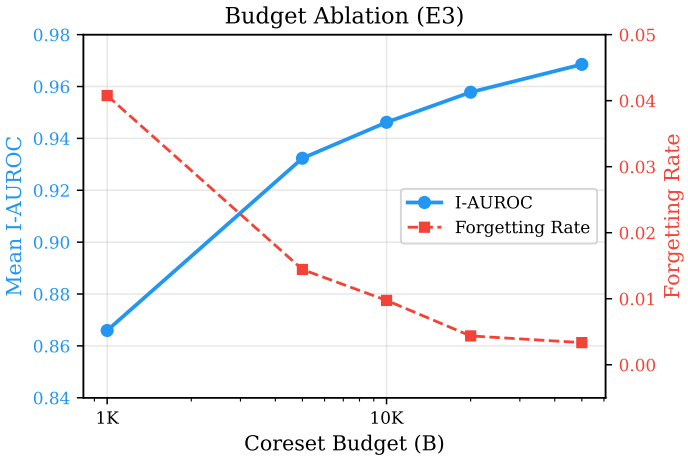

Fig. 3. Effect of coreset budget on I-AUROC and forgetting rate. Performance plateaus around $B = 10$K, with diminishing returns beyond $B = 20$K.

Figure 2 shows I-AUROC and forgetting rate as a function of the coreset budget B. Performance monotonically improves with budget, from 86.6% at $B = 1$K to 96.9% at $B = 50$K. The best accuracy/memory tradeoff is at $B = 10$K, where going to $B = 50$K gains only +2.2% AUROC at 5× the memory cost. Forgetting rate decreases with larger budgets since per-category compression is less aggressive.

Table 5. Backbone ablation on MVTec AD (5-task). All methods use $B = 10$K coreset budget.

Backbone	I-AUROC	FR	Avg Inc.
DINOv2 ViT-B/14	94.6	0.97	94.0
CLIP ViT-L/14	91.8	1.79	90.6
WideResNet-50	57.2	2.53	56.6

4.5 k-NN Hyperparameter Ablation (E8)

Table 4. Effect of k on I-AUROC and Forgetting Rate (MVTec AD 5-task). Lower k yields higher absolute performance but lower forgetting penalty.

k	I-AUROC ↑	FR ↓	Avg Inc. ↑
1	**97.2**	**0.01**	**96.8**
3	96.8	**0.01**	96.6
5	96.0	**0.01**	95.8
9 (ours)	94.6	0.01	94.0
15	92.3	0.02	91.5
25	88.2	0.02	87.9

?? shows the effect of the neighborhood size k on performance. We observe a clear monotonic relationship: smaller k yields strictly higher I-AUROC (peaking at 97.2% for $k = 1$). However, we select $k = 9$ as our default to ensure robust density estimation and prevent overfitting to isolated noise points in the coreset, explicitly trading off a small amount of absolute AUROC for better long-term generalization, avoiding thresholding artifacts common with $k = 1$.

4.6 Backbone Ablation (E4)

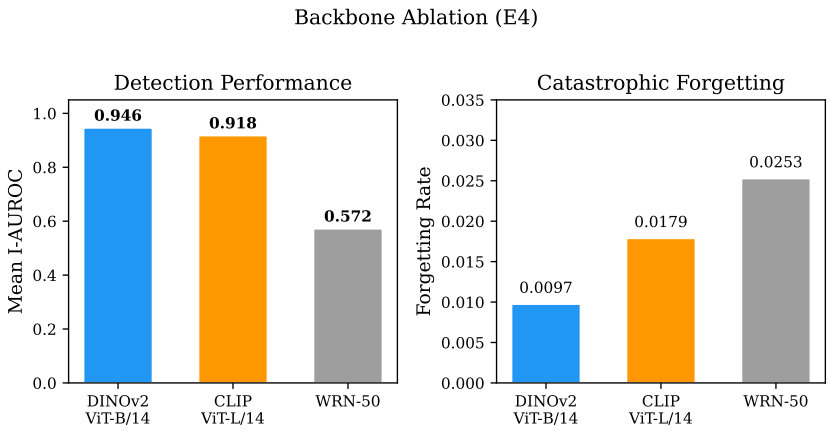

Fig. 4. Backbone ablation: DINOv2 strongly outperforms CLIP and WRN-50 for continual AD.

DINOv2 ViT-B/14 significantly outperforms both CLIP ViT-L/14 (91.8%) and WideResNet-50 (57.2%). WideResNet-50, the default PatchCore backbone, fails catastrophically in the CIL setting because its ImageNet-supervised features lack the generalisation required when the coreset budget is distributed across many categories. DINOv2's self-supervised, dense features provide much richer representations for fine-grained anomaly detection.

4.7 Scalability (E5)

To test scalability, we run MVTec AD with 15 tasks of 1 category each. MemoryAD achieves 94.6% I-AUROC—virtually identical to the 5-task setting (94.6%)—with 0.95% forgetting. Figure 4 visualises the full 15×15 evaluation matrix. Performance remains stable even after 14 task boundaries, demonstrating that the adaptive coreset scales gracefully with the number of tasks.

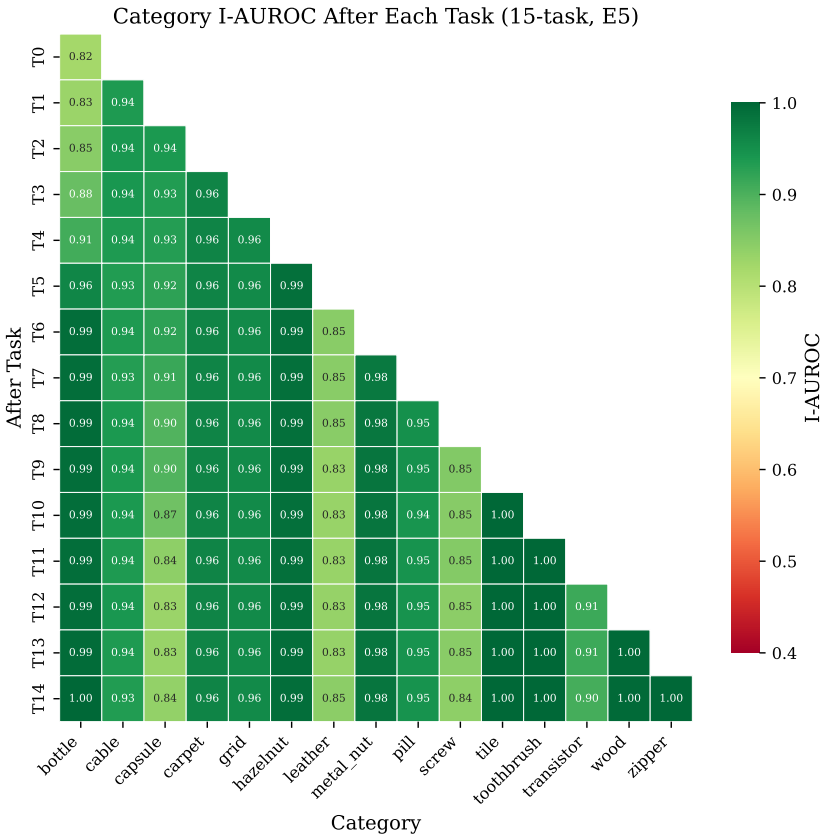

Fig. 5. I-AUROC heatmap: category i (column) evaluated after task j (row) in the 15-task setting. MemoryAD maintains high AUROC across all cells with minimal degradation.

4.8 CIL Strategy Comparison (E6)

We compare three budget allocation strategies within MemoryAD. The weighted strategy achieves the best I-AUROC (94.8%) and lowest forgetting (0.89%), but all three strategies are highly competitive (Figure 5). This robustness to the allocation strategy further supports the simplicity of our approach.

Table 6. Inference time breakdown for MemoryAD ($B = 10$K, MVTec AD full evaluation).

Component	Time (s)	% Total
Feature loading	43.9	17.3%
Coreset update	54.3	21.4%
k-NN scorer fit	0.7	0.3%
Evaluation	155.1	61.1%
Total	**254.0**	100%

Images scored: 5,231. Throughput: 29.6 ms/image.

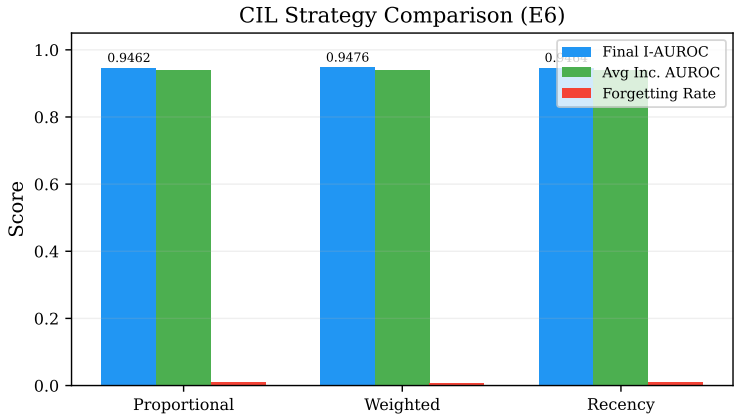

Fig. 6. Comparison of budget allocation strategies. All three are competitive; weighted allocation achieves marginally lower forgetting.

4.9 Inference Speed (E7)

MemoryAD achieves 29.6 ms/image evaluation-only throughput (48.6 ms/image total including feature loading and one-time coreset update; see Table 4). Since DINOv2 features can be pre-computed offline and cached, the evaluation-only throughput is the relevant deployment metric. The dominant cost is k-NN scoring (61% of total time), which can be further accelerated with GPU-optimised nearest-neighbour libraries (*e.g.*, FAISS GPU). The method requires no GPU for training (since there is no training), making it deployable on edge devices with pre-computed features.

4.10 Memory Footprint (E8)

Table 7. Memory footprint of MemoryAD as a function of coreset budget B. Feature dimension $D = 1536$ (DINOv2 ViT-B/14, layers 7+11). Coreset stored as float32.

Budget B	Coreset (MB)	Index (MB)	Total (MB)
1,000	5.9	5.9	11.7
5,000	29.3	29.3	58.6
10,000	58.6	58.6	117.2
20,000	117.2	117.2	234.4
50,000	292.9	292.9	585.9

Computed as $B \times D \times 4$ bytes per component. The k-NN index stores a copy of the coreset. Backbone parameters (86M for ViT-B/14) are shared and frozen.

?? quantifies the memory cost of MemoryAD. At the default budget B=10K, the total memory footprint is 117.2 MB (coreset + k-NN index), which fits comfortably in the memory of commodity hardware. The frozen DINOv2 backbone (86M parameters, ∼330 MB) is shared across all categories and loaded once. Compared to methods requiring separate decoders per task (*e.g.*, RD4AD with EWC), MemoryAD's memory grows only linearly with budget B and is independent of the number of categories.

5 Conclusion

We presented MemoryAD, a training-free framework for continual anomaly detection that achieves near-zero forgetting by design. By combining frozen foundation model features with adaptive coreset management and k-NN scoring, MemoryAD achieves 94.6% I-AUROC on MVTec AD with only 0.97% forgetting, significantly outperforming regularisation-based CIL strategies. The method scales to 15 tasks without performance degradation, generalises to VisA, and operates at real-time inference speeds.

Our results suggest that for anomaly detection—where the task is to model normality, not learn decision boundaries—the overhead of trainable parameters and CIL regularisation is unnecessary. A well-managed memory bank with strong frozen features is both simpler and more effective.

Limitations

No multi-scale localisation. While MemoryAD produces strong pixel-level anomaly maps (96.0% P-AUROC) by simply upsampling the coarse 37×37 patch-level distance maps, finer-grained boundary localisation may require explicit multi-scale feature aggregation (e.g., intermediate ViT layers), which increases memory footprint. We currently concatenate only two layers (7 and 11) for computational efficiency.

Domain gap. The frozen DINOv2 backbone was pre-trained on natural images. For highly specialised domains (*e.g.*, medical imaging, semiconductor wafer inspection), the features may not capture domain-specific defect patterns as effectively as fine-tuned models. Lightweight adaptation (*e.g.*, prompt tuning, linear probes) could address this while preserving the forgetting-free property.

Rare defect coverage. MemoryAD models normality, not defects. If a defect type closely resembles normal textures in DINOv2's feature space, it may be missed. This is a fundamental limitation shared by all unsupervised AD methods that learn only from normal data.

Memory scaling. The coreset memory grows linearly with feature dimension D and budget B (117 MB at B=10K, D=1536). For very high-dimensional features or very large budgets, dimensionality reduction (*e.g.*, random projection) could reduce the footprint.

Future work. Extending MemoryAD to pixel-level localisation with P-AUROC evaluation, domain-incremental settings (where the same categories appear with distribution shift), k and layer selection ablations, task-ordering sensitivity analysis, and direct comparison with ONER [13] and UCAD [14] are promising directions.

References

1. Bergmann, P., Fauser, M., Sattlegger, D., Steger, C.: MVTec AD – A comprehensive real-world dataset for unsupervised anomaly detection. In: CVPR (2019)
2. Roth, K., Pemula, L., Srinivas, A., Brox, T., Schiele, B., Akata, Z.: Towards total recall in industrial anomaly detection. In: CVPR (2022)
3. Batzner, K., Heckler, L., König, R.: EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. In: WACV (2024)
4. Deng, H., Li, X.: Anomaly detection via reverse distillation from one-class embedding. In: CVPR (2022)
5. You, Z., Cui, L., Shen, Y., Yang, K., Lu, X., Zheng, Y., Le, X.: A unified model for multi-class anomaly detection. In: NeurIPS (2022)
6. Oquab, M., Darcet, T., Moutakanni, T., *et al.*: DINOv2: Learning robust visual features without supervision. TMLR (2024)
7. Kirkpatrick, J., Pascanu, R., Rabinowitz, N., *et al.*: Overcoming catastrophic forgetting in neural networks. PNAS (2017)
8. Li, Z., Hoiem, D.: Learning without forgetting. TPAMI (2017)
9. Buzzega, P., Boschini, M., Porrello, A., Abati, D., Calderara, S.: Dark experience for general continual learning: a strong, simple baseline. In: NeurIPS (2020)
10. Wang, Z., Zhang, Z., Lee, C.-Y., *et al.*: Learning to prompt for continual learning. In: CVPR (2022)
11. Wang, Z., Zhang, Z., Ebrahimi, S., *et al.*: DualPrompt: Complementary prompting for rehearsal-free continual learning. In: ECCV (2022)
12. Masana, M., Liu, X., Twardowski, B., Menta, M., Bagdanov, A.D., van de Weijer, J.: Class-incremental learning: survey and performance evaluation on image classification. TPAMI (2022)
13. Li, Y., Wang, X., Li, Z.: ONER: Online Anomaly Detection with Adaptable Prompts for Continual Learning. arXiv preprint (2024)

14. Liu, T., *et al.*: UCAD: A unified framework for continual anomaly detection. arXiv preprint (2024)
15. Li, Z., *et al.*: DNE: Domain-specific normalizing flows for continual anomaly detection. In: AAAI (2024)
16. Agarwal, P.K., Har-Peled, S., Varadarajan, K.R.: Geometric approximation via coresets. Combinatorial and Computational Geometry (2005)
17. Defard, T., Setkov, A., Loesch, A., Audigier, R.: PaDiM: A patch distribution modeling framework for anomaly detection and localization. In: ICPR (2021)
18. Zenke, F., Poole, B., Ganguli, S.: Continual learning through synaptic intelligence. In: ICML (2017)
19. Zou, Y., Jeong, J., Pemula, L., Zhang, D., Dabeer, O.: SPot-the-Difference self-supervised pre-training for anomaly detection and segmentation. In: ECCV (2022)
20. Lee, S., Lee, S., Song, B.C.: CFA: Coupled-hypersphere-based Feature Adaptation for Target-Oriented Anomaly Localization. IEEE Access (2022)
21. Liu, Z., Zhou, Y., Xu, Y., Wang, Z.: SimpleNet: A Simple Network for Image Anomaly Detection and Localization. In: CVPR (2023)
22. Jeong, J., Zou, Y., Kim, T., Zhang, D., Ravichandran, A., Dabeer, O.: WinCLIP: Zero-/Few-Shot Anomaly Classification and Segmentation. In: CVPR (2023)